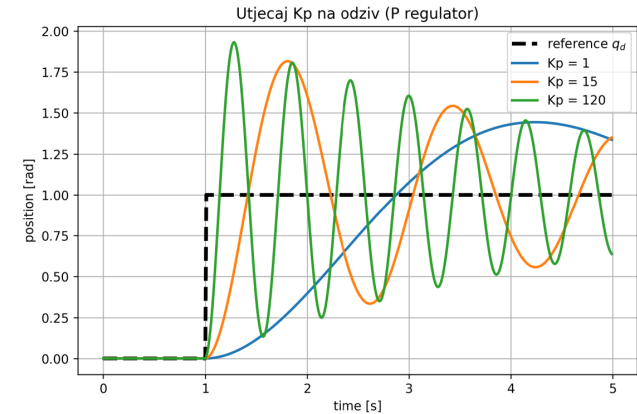
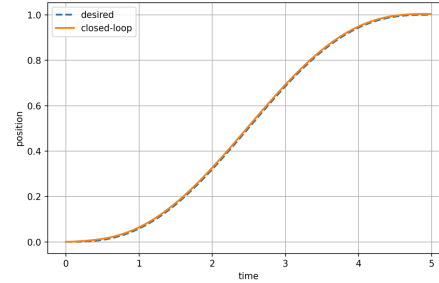
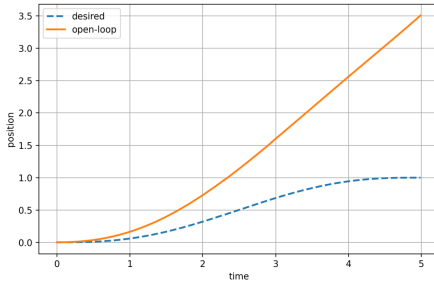
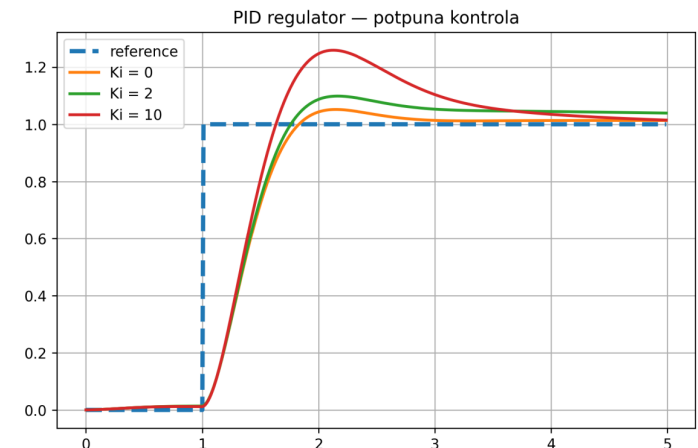
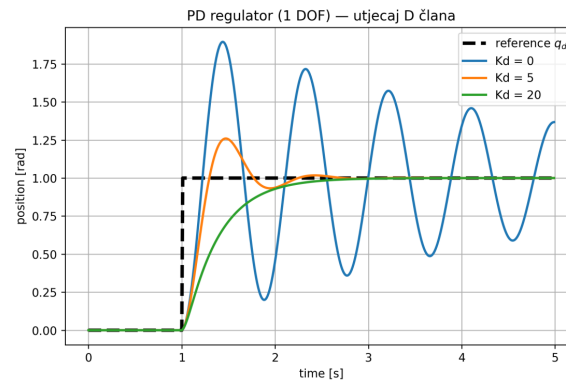
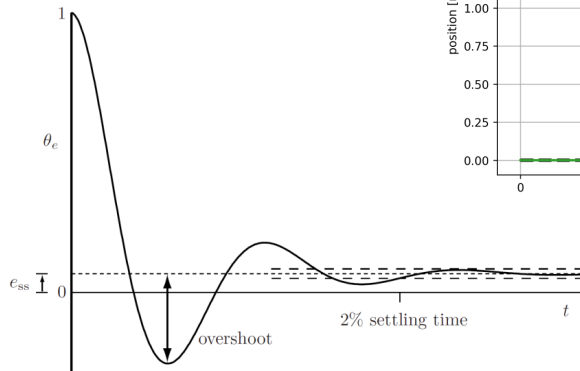


Uvod u upravljanje robotom: PID kontrola i praćenje trajektorije



Tara Knežević,
mag. ing. mech.



Generiranje trajektorija - podsjetnik



- Koji je zakon gibanja (pozicija, brzina, akceleracija) između zadanih stanja?
 - Putanja $\neq$ trajektorija
 - Trajektorija kao interpolacija između početnog i završnog stanja
-
- Što čini dobru trajektoriju? (C^2 kontinuiranost, poštivanje ograničenja, izvedivost (singulariteti))
 - Gdje planiramo? (joint space $\rightarrow$ lakše upravljanje, task space $\rightarrow$ intuitivnije, ali složenije)
 - Prošli put smo odgovorili na pitanje **KAKO** želimo da se robot giba

Današnje pitanje: “Kako natjerati robota da stvarno prati tu trajektoriju?”

- **Humanoid Touch Dream** – roboti koji razumiju dodir i kontakt
- Iz perspektive upravljanja: ovakvi sustavi zahtijevaju kontinuiranu prilagodbu upravljačkih signala na temelju senzorskog feedbacka, a ne samo praćenje unaprijed zadane trajektorije

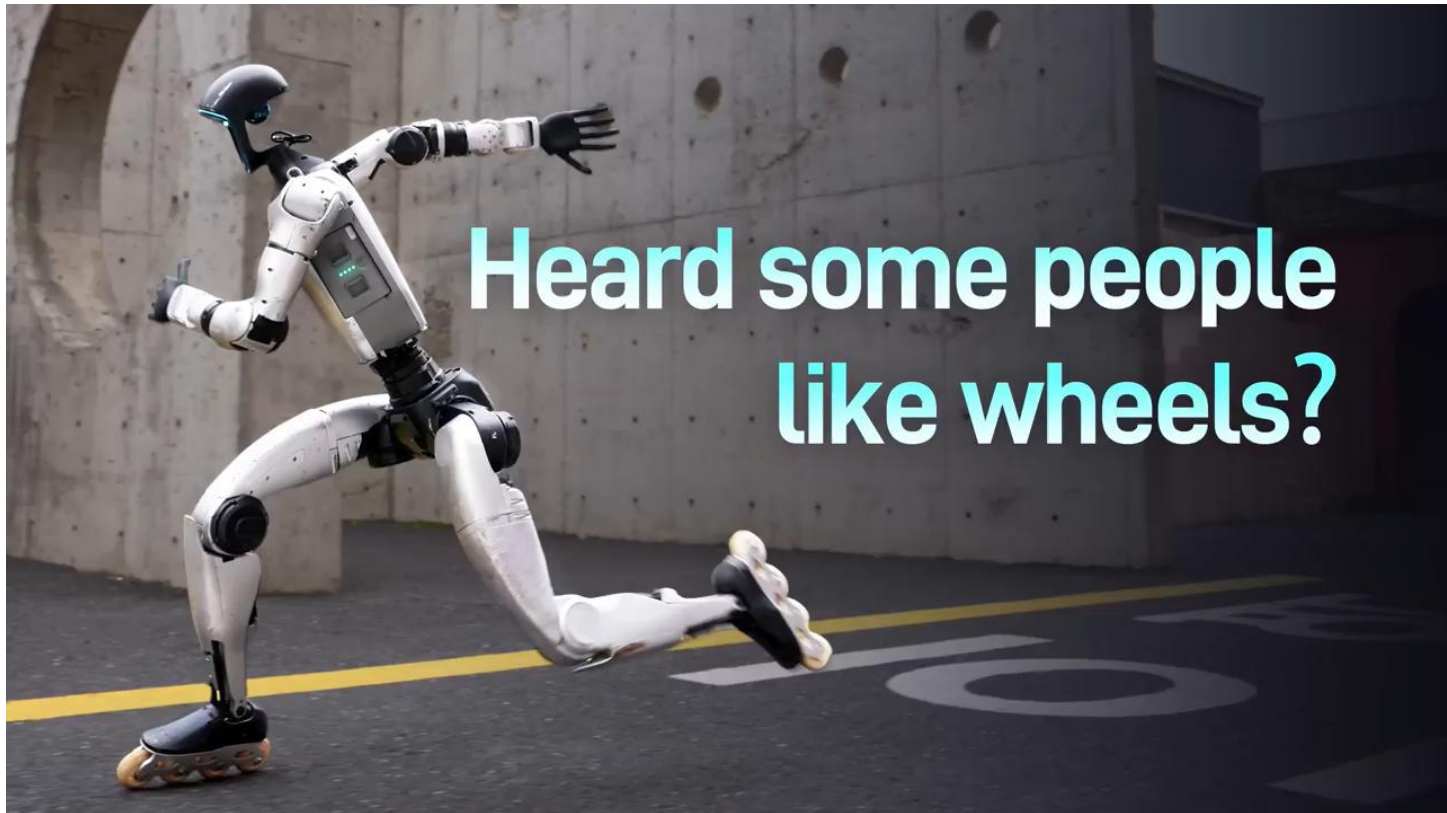


- koriste tactile sensing
- predviđaju sile u kontaktu
- uče iz podataka

<https://humanoid-touch-dream.github.io/>

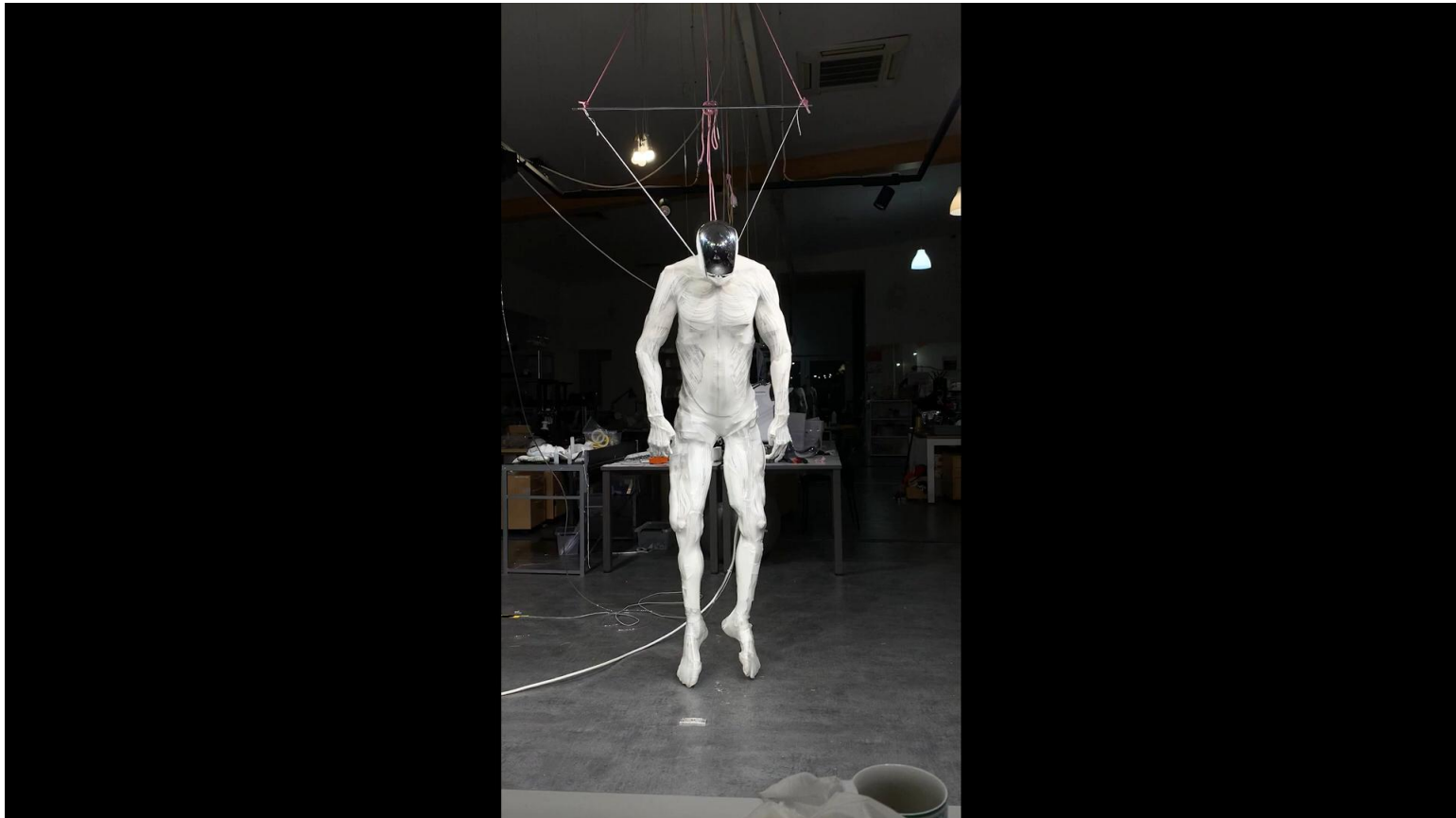
Današnje pitanje: “Kako natjerati robota da stvarno prati tu trajektoriju?”

- **Unitree Robotics** – whole body motion control
- Iz perspektive upravljanja: svi zglobovi sustava koordinirano upravljaju s ciljem ostvarivanja željenog gibanja uz istovremeno održavanje stabilnosti, dinamičkih ograničenja i kontakta s okolinom



Današnje pitanje: “Kako natjerati robota da stvarno prati tu trajektoriju?”

- **Clone Robotics** – bioinspirirana arhitektura mišića
- Iz perspektive upravljanja: veoma nelinearni sustavi, teški za modeliranje, zahtijevaju napredne metode upravljanja



Današnje pitanje: “Kako natjerati robota da stvarno prati tu trajektoriju?”

- **Clone Robotics** – bioinspirirana arhitektura mišića
- Iz perspektive upravljanja: veoma nelinearni sustavi, teški za modeliranje, zahtijevaju napredne metode upravljanja



Osnove upravljanja robotskim sustavima

OD PLANIRANJA DO UPRAVLJANJA

Poveznica planiranje $\leftrightarrow$ upravljanje

- Planiranje (generiranje trajektorija) definira željeno gibanje:

$$q_d(t), \dot{q}_d(t), \ddot{q}_d(t)$$

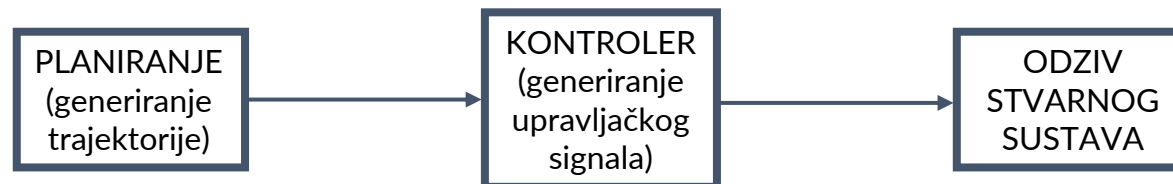
- Stvarni sustav (robot) ima dinamiku, trenja, ograničenja, zato $q(t) \neq q_d(t)$

$$\ddot{q} = f(q, \dot{q}, \tau)$$

Zašto koristimo direktnu
dinamiku?

- Upravljanje (control) generira upravljački signal $u(t)$ ili $\tau(t)$, kojem je cilj:

$$q(t) \approx q_d(t)$$



Kontroler ispravlja razliku između plana i stvarnosti

Open-loop control

- Osnovna ideja ovakvog upravljanja je:

$$u(t) = \ddot{q}_d(t)$$

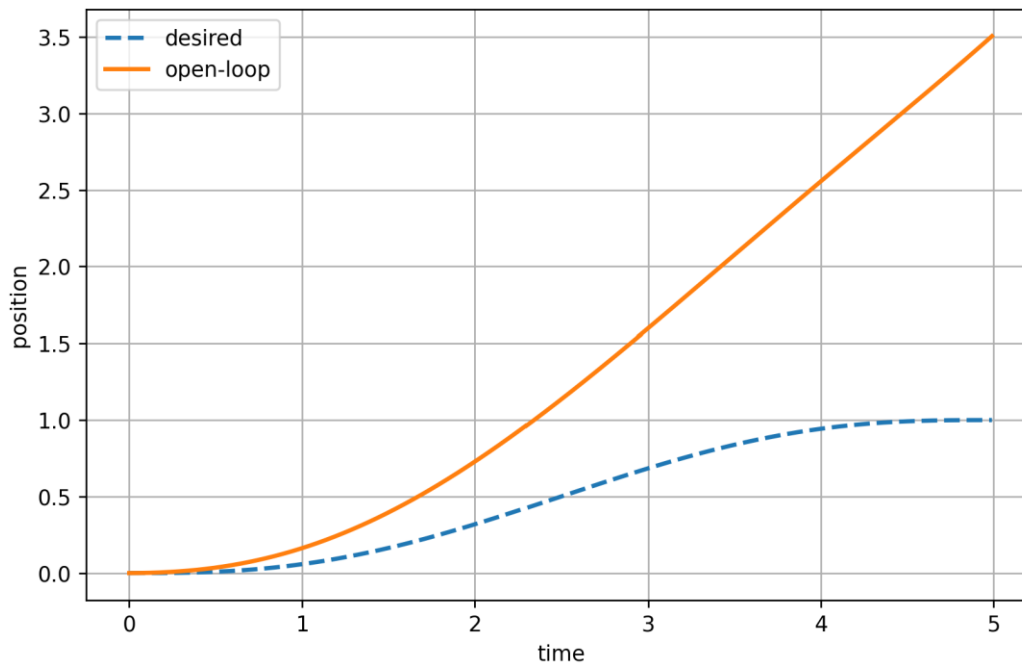
Upravljački signal generiramo samo iz planirane trajektorije, bez mjerenja stvarnog stanja

- Što to stvarno znači:
 - Robot NE zna gdje se stvarno nalazi
 - Nema informacije o grešci → nema korekcije greške
 - Radi samo ako je: model robota savršen, nema smetnji, nema šuma, aktuatori su idealni (realnost?)
- U open-loop pristupu samo pošaljemo plan robotu, ali robot ne provjerava je li ga stvarno pratio, zato **open-loop u praksi gotovo nikad ne radi pouzdano**

Open-loop control

- Realnost:
 - Nikad nemamo točan model robota
 - Uvijek imamo nelinearnosti (trenje)
 - Uvijek postoje vanjski utjecaji (šum)

BEZ FEEDBACKA → NEMA
KOREKCIJE GREŠKE



Primjer:

- 1 DOF
- Zadana trajektorija definirana polinomom 5. reda ($q_i = 0, q_f = 1$)
- Bez povratne veze (no feedback)
- Smetnja (disturbance) modelirana kao konstantni dodatni utjecaj

$$\ddot{q} = u + d$$

Closed-loop control (feedback)

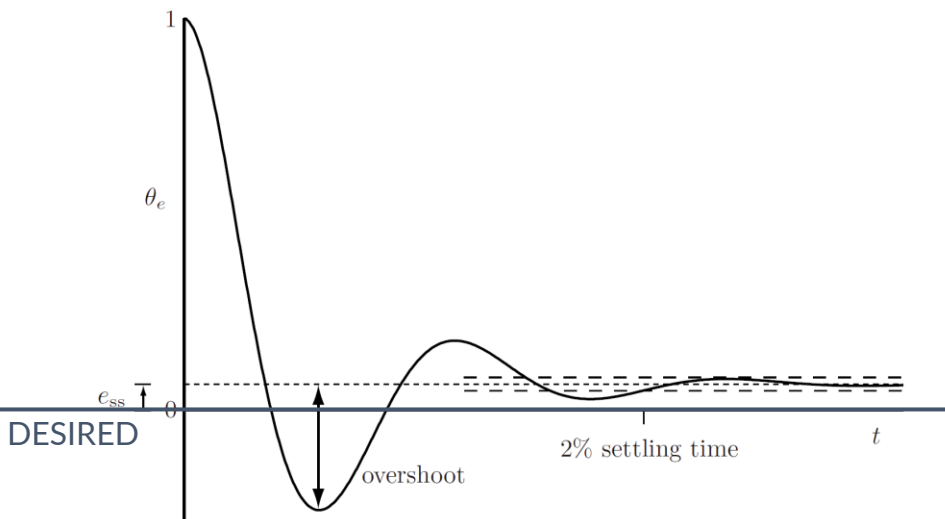
- Osnovna ideja ovakvog upravljanja je:

$$e(t) = q_d(t) - q(t), e(t) \rightarrow 0$$

kontroler koristi grešku za upravljanje, te ju je cilj minimizirati

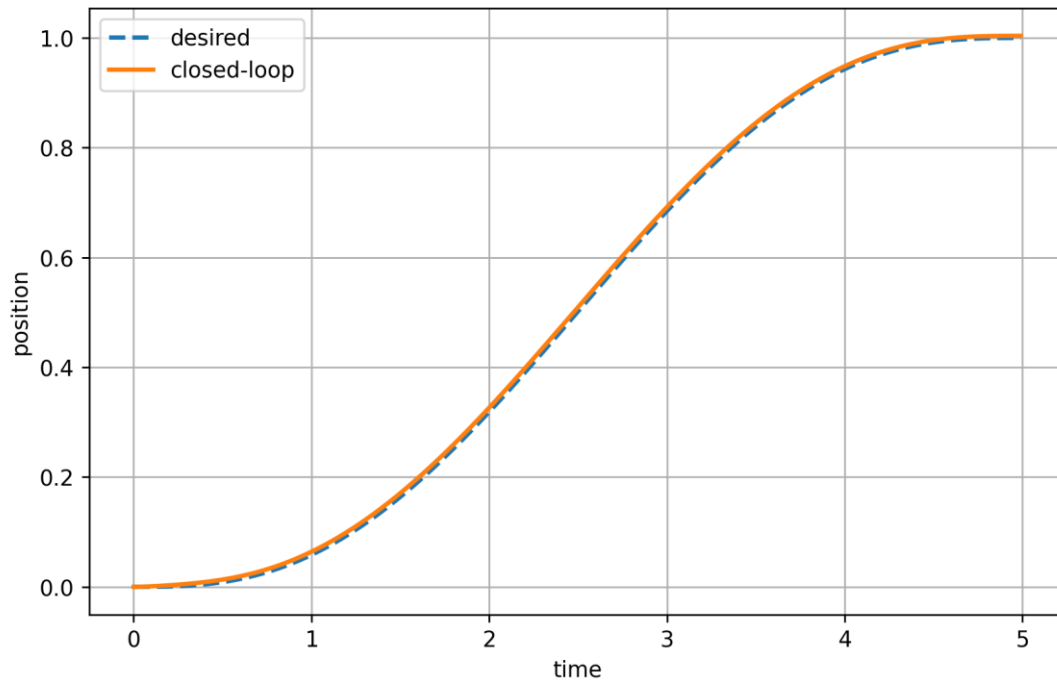
- Razlika u odnosu na open-loop je da sustav:
 - Mjeri stvarno stanje
 - Uspoređuje stvarno stanje sa željenim
 - Ispravlja grešku u realnom vremenu

Što je bolje? Da sustav nikad ne oscilira ili da brzo dođe do cilja?



- Overshoot = koliko sustav prijeđe cilj
- Oscilacije = titranje oko ciljne vrijednosti
- Settling time = vrijeme do smirivanja u ciljnom području
- Steady-state error = razlika od cilja nakon smirivanja

Closed-loop control (feedback)



Primjer:

- 1 DOF
- Zadana trajektorija definirana polinomom 5. reda ($q_i = 0, q_f = 1$)
- S povratnom vezom (PD regulator)
- Smetnja (disturbance) modelirana kao konstantni dodatni utjecaj

$$\ddot{q} = u + d$$

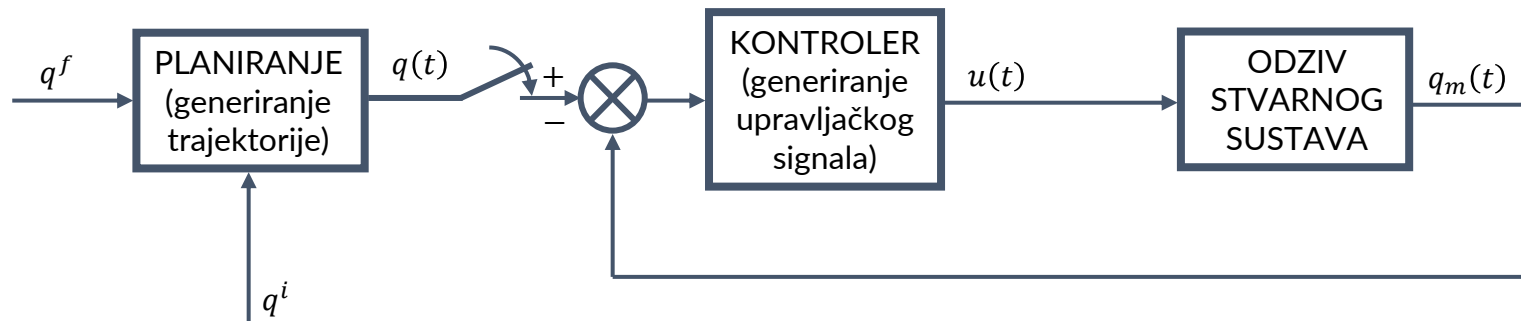
FEEDBACK OMOGUĆUJE ROBUSNO
UPRAVLJANJE U REALNIM UVJETIMA

Pregled upravljačkog sustava

Što se stvarno događa u robotu?

- Upravljanje nije samo ovisno o upravljačkom zakonu, bitne komponente su:
 - Senzori – mjerenje stanja
 - **Kontroleri** – računa upravljački signal
 - Aktuatori i prijenos – generiraju sile/momente
 - Dinamika robota i okoline – stvarno gibanje

Pojednostavljen model zatvorene petlje (koji ćemo mi koristiti)



Kako konkretno iz greške generirati upravljački signal?

Osnovne metode upravljanja: P, PD i PID

OD GREŠKE DO UPRAVLJAČKOG SIGNALA

Proporcionalni (P) regulator:

- Referencu (generiranu trajektoriju) dobijemo planiranjem: $q_d(t)[rad]$
- Mjerimo stvarno stanje robota: $q_m(t)[rad]$
- Definiramo grešku: $e(t)[rad] = q_d(t) - q_m(t)$
- **Regulator iz greške generira upravljački signal:**

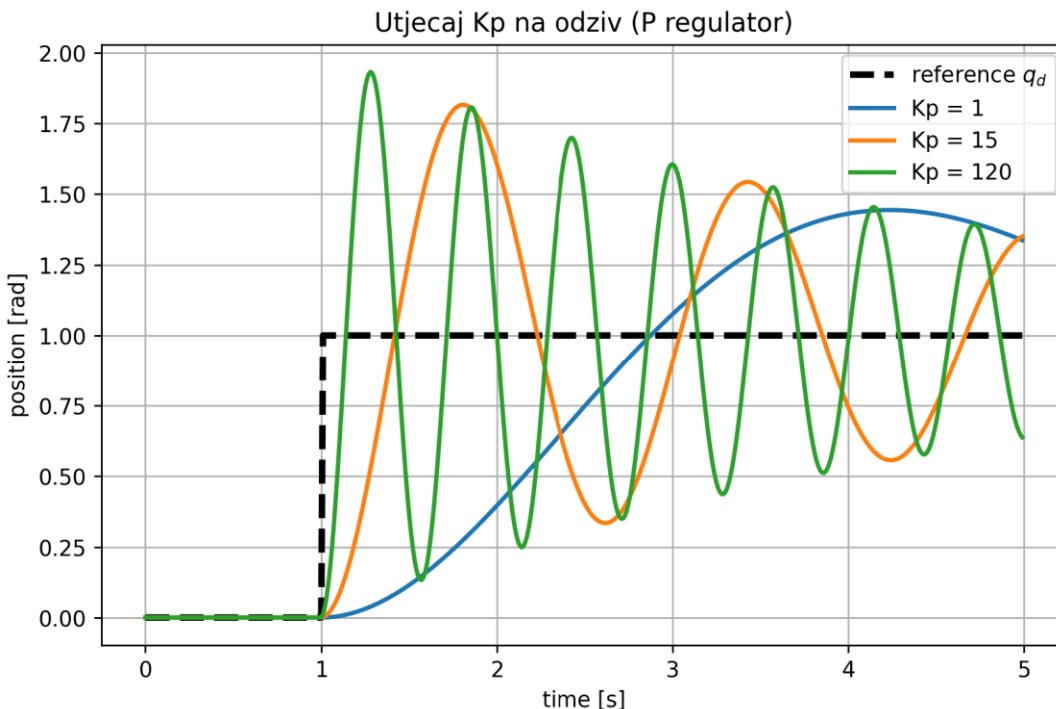
$$u(t)[Nm] = K_p \left[\frac{Nm}{rad} \right] e(t)$$

↳ UPRAVLJAČKI SIGNAL = komanda koju šaljem aktuatorima. Regulator ne daje poziciju, nego silu/moment koji mijenja gibanje

- Intuicija (ponašanje regulatora):
 - veća greška → veći signal prema robotu (moment)
 - manja greška → manji signal prema robotu (moment)

Utjecaj parametra K_p na ponašanje sustava

- Što se događa kad mijenjamo K_p ? Primjer: step pobuda, bez vanjskih smetnji



PARAMETRI REGULATORA UTJEČU NE SAMO
NA BRZINU, NEGO I NA STABILNOST SUSTAVA

Mali K_p :

- spor odziv (dugo traje)
- sustav polako dolazi do cilja
- velika razlika $q_d(t) - q_m(t)$

Veliki K_p :

- brži odziv
- može doći do overshoot-a
- moguće oscilacije

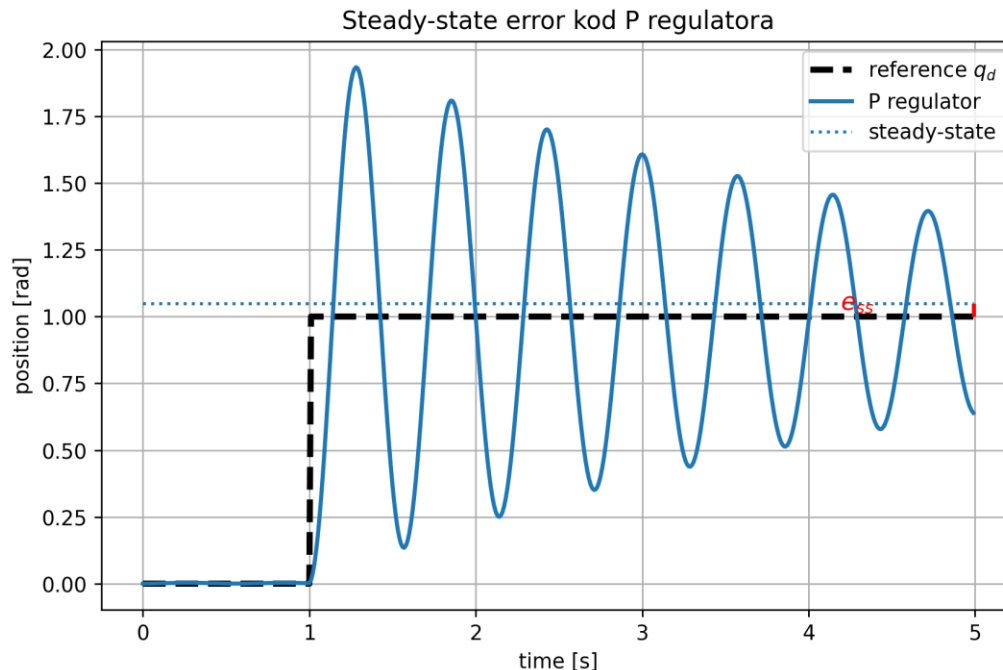
Stabilnost:

- Sustav je stabilan ako vrijedi:

$$e(t) \rightarrow 0 \text{ kad } t \rightarrow \infty$$

Utjecaj parametra K_p na ponašanje robota

- Što se događa kad mijenjamo K_p ? Primjer: step pobuda, konstantna vanjska smetnja



P REGULATOR SAM PO SEBI NE MOŽE
UKLONITI STEADY-STATE GREŠKU

Steady-state error:

- U odnosu na prošli primjer, dodali smo i smetnju (disturbance)
- Sustav pokušava doći do 1 (reference), ali ga smetnja stalno „gura” u nekom smjeru
- P regulator reagira samo na trenutnu grešku, nastaje ravnoteža:

$$K_p e_{ss} = \text{smetnja}$$

- U našem primjeru:

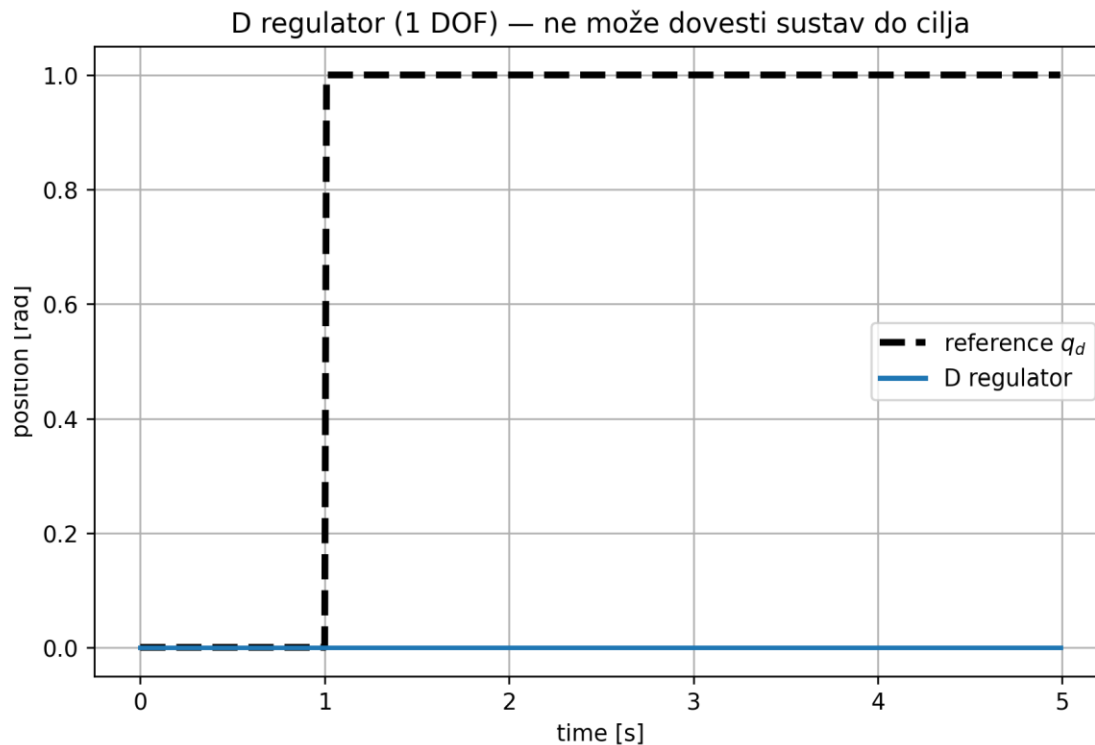
$$K_p = 120, \text{smetnja} = 0.2$$

Derivacijski (D) regulator:

- Definicija: $u(t) = K_d \left[\frac{Nm\ s}{rad} \right] \cdot \dot{e}(t)$, gdje vrijedi: $\dot{e}(t) = \frac{d}{dt}[q_d(t) - q_m(t)]$
- Intuicija (ponašanje regulatora):
 - „predviđamo budućnost” → pratimo trend promjene greške
 - ako se sustav brzo kreće prema cilju → koči
 - ako se sustav sporo kreće prema cilju → ubrzaj
- Ovaj član smatra se i **prigušenjem** (svojim djelovanjem sprječava/smanjuje oscilacije izazvane visokim K_p) jer stabilizira odziv
- D član reagira na brzinu promjene greške (prigušenje/damping)

Utjecaj parametra K_d na ponašanje sustava

- Što se događa kad koristimo samo D član? Primjer: *step pobuda*, bez vanjskih smetnji



U našem primjeru: $K_d = 20$

Ponašanje sustava:

- ne ide prema cilju
- može malo reagirati na step
- nikad ne dosegne referencu

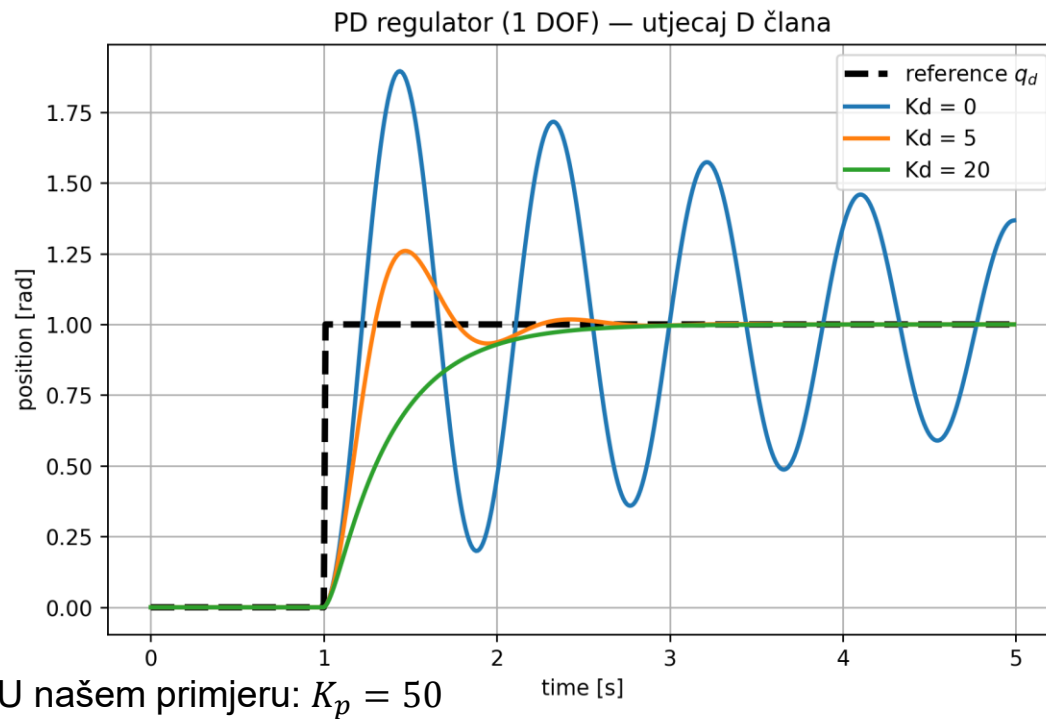
ZAŠTO?

Jer derivacijski član reagira samo na promjenu greške, a ne na samu grešku

D ČLAN SAM NE MOŽE UPRAVLJATI
SUSTAVOM — TREBA P ČLAN

PD regulator – kombinacija P i D članova

- Što se događa kad mijenjamo K_d ? Primjer: *step pobuda*, uz konstantan K_p , bez vanjskih smetnji



D ČLAN NE VODI SUSTAV PREMA CILJU – NEGO KONTROLIRA KAKO DOLAZIMO DO NJEGA, PD REGULATOR SAM PO SEBI NE MOŽE RIJEŠITI STEADY-STATE GREŠKU

$K_d = 0$:

- prisutan overshoot
- izražene oscilacije
- sporije smirivanje sustava

Mali K_d :

- smanjenje oscilacija
- blaži overshoot
- sustav se brže smiruje

Veliki K_d :

- snažno prigušenje
- bez overshoota
- sporiji odziv

Stabilnost:

- D član povećava stabilnost sustava
- omogućuje veći K_p bez nestabilnosti

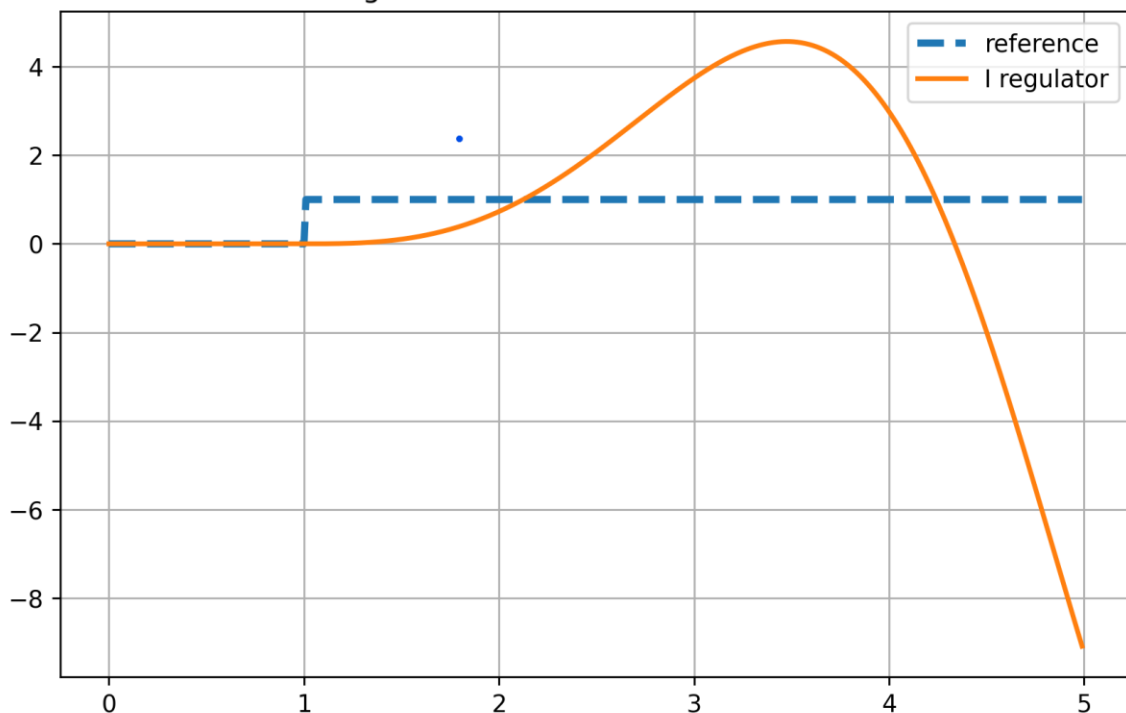
Integralni (I) regulator:

- Definicija: $u(t) = K_i \left[\frac{Nm}{rad\ s} \right] \int e(t) dt$
- Intuicija (ponašanje regulatora):
 - I član akumulira grešku kroz vrijeme
 - ako greška postoji dugo $\rightarrow$ upravljački signal raste
- Ovaj član uklanja steady-state error
- Ako sustav stalno malo griješi $\rightarrow$ I član to “pamti”, a zatim sve više “gura” sustav prema cilju
- Integral windup – integral može „narasti previše”
 - Izaziva oscilacije i nestabilnosti u odzivu sustava

Utjecaj parametra K_i na ponašanje sustava

- Što se događa kad koristimo samo I član? Primjer: step pobuda, bez vanjskih smetnji

I regulator — neadekvatan samostalno



U našem primjeru: $K_i = 5$

Ponašanje sustava:

- sustav ide prema cilju
- odziv je spor
- javljaju se velike oscilacije
- sustav može postati nestabilan

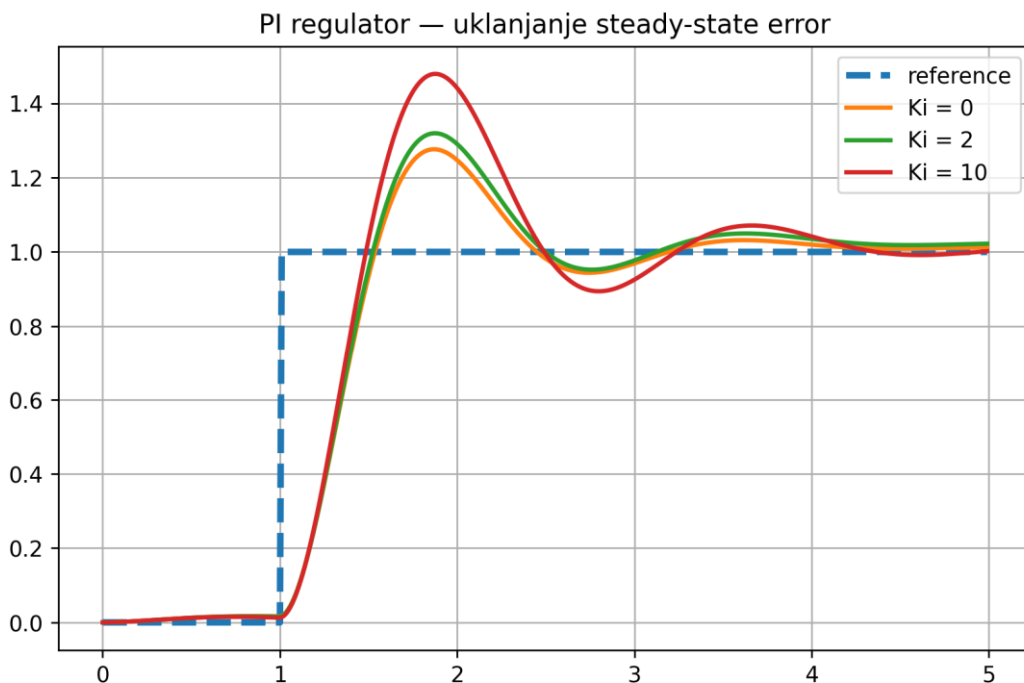
ZAŠTO?

Integralni član akumulira grešku kroz vrijeme. Što dulje greška traje, signal postaje sve veći.

I ČLAN UKLANJA GREŠKU, ALI MOŽE
'PRETJERATI' JER PAMTI PROŠLOST

Utjecaj PI regulatora na ponašanje sustava

- Što se događa kad kombiniramo P i I član? Primjer: step pobuda, uz konstantnu smetnju



U našem primjeru: $K_p = 15$

PI REGULATOR UKLANJA TRAJNU GREŠKU,
ALI MOŽE NARUŠITI STABILNOST

Ponašanje sustava:

- sustav ide prema cilju
- dolazi točno na referencu
- uklanja steady-state error
- može imati overshoot i oscilacije

ZAŠTO?

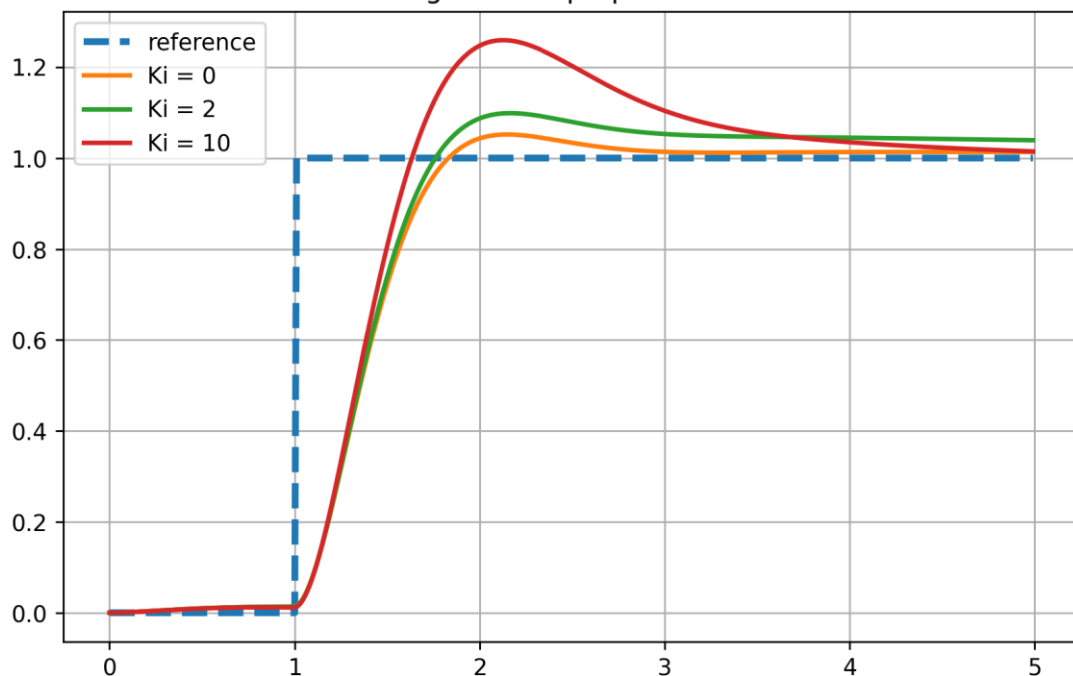
P član reagira na trenutnu grešku, a I član akumulira grešku kroz vrijeme.
P → vuče sustav prema cilju
I → “gura dodatno” ako greška traje

Ključna prednost PI regulatora je eliminacija steady-state errora

Utjecaj PID regulatora na ponašanje sustava

- Što dobivamo kada kombiniramo P, I i D član? Primjer: step pobuda, uz konstantnu smetnju

PID regulator — potpuna kontrola



U našem primjeru: $K_p = 15$, $K_d = 5$

NAJBOLJI KOMPROMIS: BRZINA + STABILNOST + TOČNOST

Ponašanje sustava:

- sustav brzo dolazi prema cilju
- dolazi točno na referencu
- minimalan steady-state error
- smanjene oscilacije (u odnosu na PI)
- kontroliran overshoot

Svaki član ima svoju ulogu:

P → reagira na trenutnu grešku

I → uklanja trajnu grešku

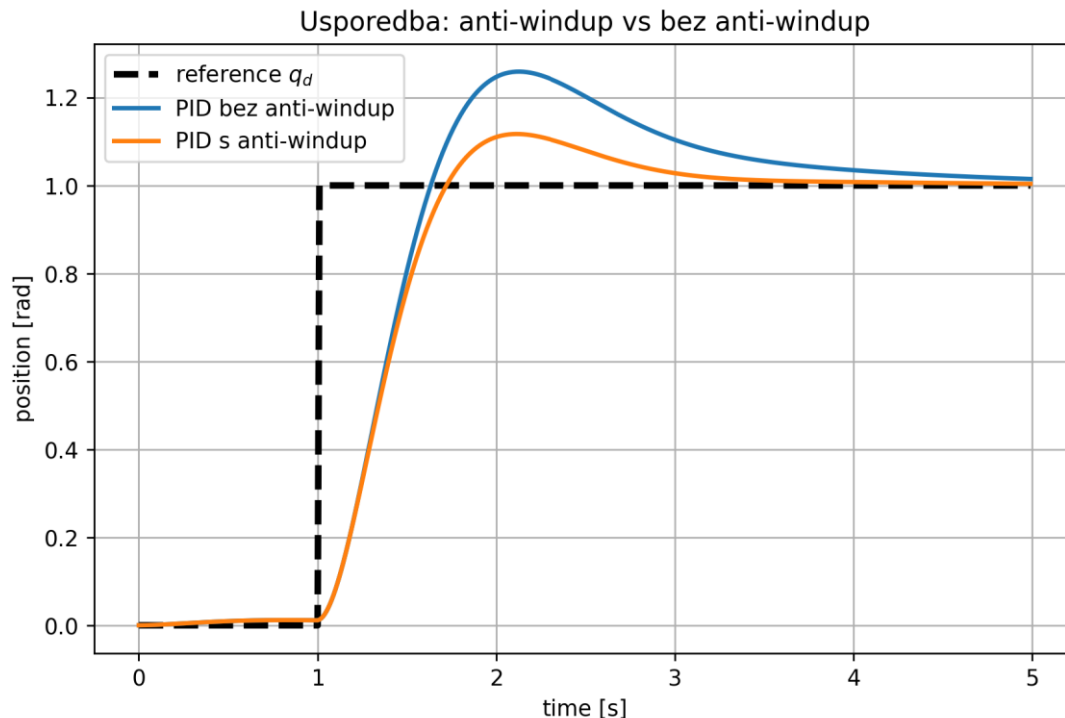
D → prigušuje gibanje

Potencijalni problemi:

- loše podešeni parametri → oscilacije
- prevelik K_i → **windup**
- osjetljivost D člana na šum

Anti-windup – ograničenje integralnog člana

- Problem: Integralni član može rasti bez ograničenja



U našem primjeru: $K_p = 15$, $K_d = 5$, $K_i = 10$, integral limit = 0.1

INTEGRAL JE KORISTAN, ALI MORA BITI OGRANIČEN

Ponašanje sustava:

- akumulira grešku kroz vrijeme
- upravljački signal postaje prevelik
- dolazi do nestabilnosti / velikog overshoot-a
- problem nastaje kad je **aktuator saturiran**

ZAŠTO?

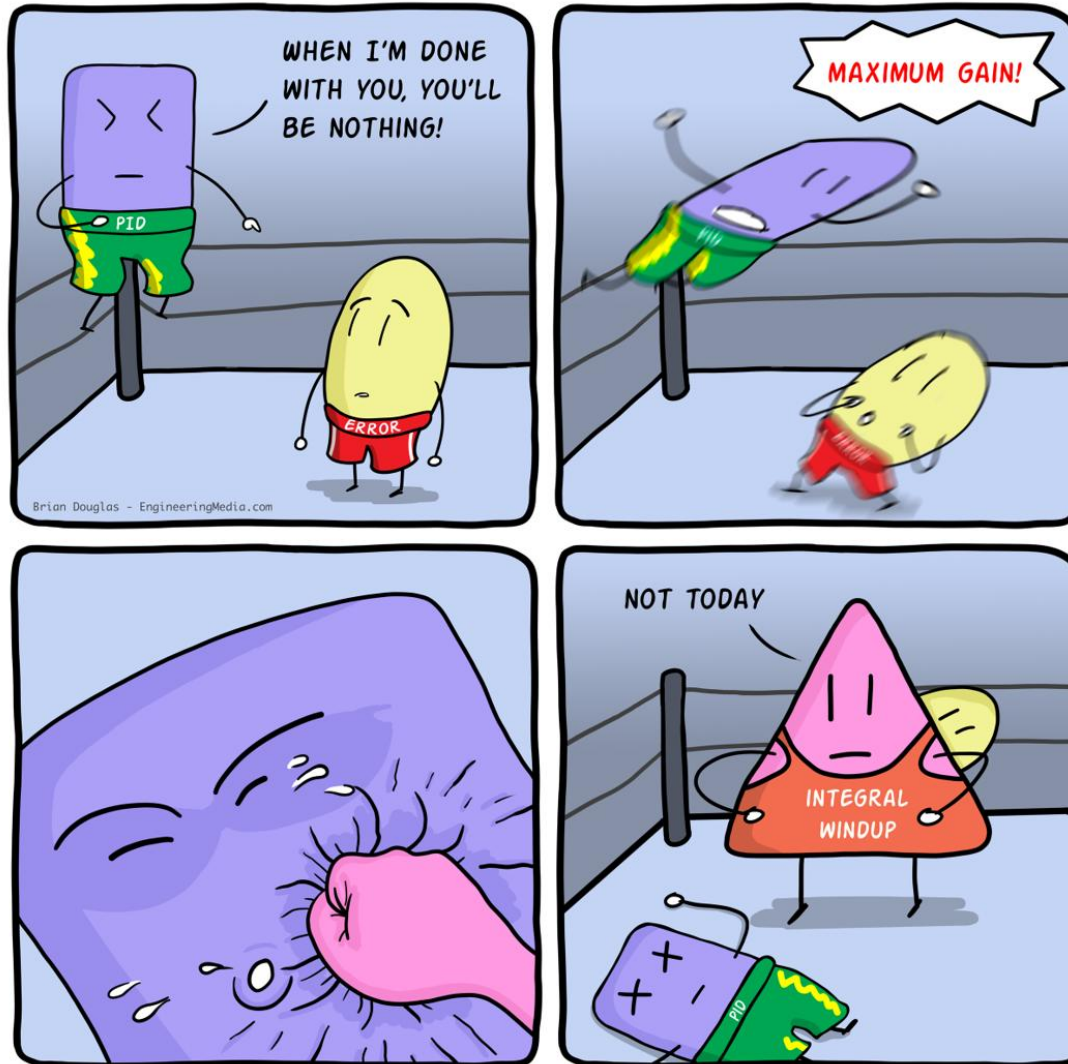
Regulator “pamti” staru grešku čak i kad više nije relevantna.

Rješenje:

- Ograničavamo integralni član na određenu vrijednost:

$$\int e(t) dt \rightarrow clip(\cdot)$$

Anti-windup: INTEGRAL JE KORISTAN, ALI MORA BITI OGRANIČEN



Task space

Gdje definiramo gibanje?

- Definiramo grešku: $e(t)[m] = x_d(t) - x_m(t)$
 - Bitno za sjetiti se: $x = f(q)$ - ne upravljamo direktno s x , nego preko q
 - Upravljački zakoni (P, PD, PID) definiraju se analogno kao u joint space-u, ali koriste grešku u task space-u
1. P regulator: $\dot{x} = K_p(x_d - x_m)$
 2. PD regulator: $\dot{x} = K_p(x_d - x_m) + K_d(\dot{x}_d - \dot{x}_m)$
 3. PID regulator: $\dot{x} = K_p(x_d - x_m) + K_d(\dot{x}_d - \dot{x}_m) + K_i \int (x_d - x_m) dt$
- Nakon definiranja upravljačke komande preko greške, želimo dobiti upravljačku komandu u joint space: $\dot{q} = J^{-1}\dot{x}$ ili $\dot{q} = J^+\dot{x}$

PID DAJE SIGNAL U TASK SPACE-U KOJI SE ZATIM
MAPIRA U JOINT SPACE PREKO JACOBIANA

Podešavanje parametara regulatora (PID tuning)

SINTEZA I PODEŠAVANJE REGULATORA

Pregled metoda podešavanja regulatora

1. Empirijske (heurističke) = temeljene na eksperimentu i pravilima
 2. Model-based = koriste matematički model sustava
 3. Optimizacijske = minimiziraju neku funkciju greške
 4. Auto-tuning = sustav sam podešava parametre
 5. Ostalo (frekvencijske metode, manual tuning)
- **Različite metode = različiti kompromisi (brzina, stabilnost, robusnost)**

Metrike

- Bitne su jer govore o kvaliteti odziva i odabranih parametara
- Nije svaka metrika prigodna svakoj referentnoj trajektoriji – potrebno je znati koju kada koristiti

1. Overshoot = mjera agresivnosti regulatora, govori nam koliko sustav „prebaci” referencu

$$OS = \frac{q_{max} - q_{ref}}{q_{ref}} \cdot 100\%$$

- q_{max} = maksimalna vrijednost odziva
 - q_{ref} = željena (referentna) vrijednost
- Mali OS = stabilan, ali spor odziv
 - Veliki OS = brz, ali oscilatoran odziv
 - Kada koristiti?
 - Kod reference s nekom konačnom vrijednošću (npr. step odziv)
 - NE! Kod referenci koje prate neke profile trajektorija (npr. sinusoida)

Metrike

2. Settling time = vrijeme potrebno da sustav “uđe i ostane” u toleranciji

$$t_s = \min\{t: |q(t) - q_{ref}| < \epsilon |q_{ref}|\}$$

- Tipično, $\epsilon = 0.02$ (2%)
- Mali t_s = brz sustav
- Veliki t_s = spor sustav
- Kada koristiti?
 - Kod reference s nekom konačnom vrijednošću (npr. step odziv)
 - NE! Kod referenci koje prate neke profile trajektorija (npr. sinusoida)

3. Steady-state error = koliko sustav promaši cilj u stacionarnom stanju

$$e_{ss} = q_{ref} - q(t \rightarrow \infty)$$

- $e_{ss} = 0$, idealno praćenje
- $e_{ss} \neq 0$, bias u sustavu
- Kada koristiti?
 - Kod reference s nekom konačnom vrijednošću (npr. step odziv)
 - Kod referenci koje prate neke profile trajektorija (npr. sinusoida – kao prosječna greška)

Metrike

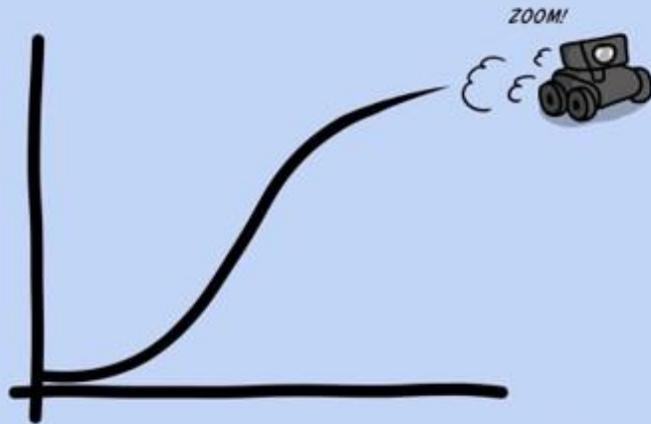
4. RMS error (tracking error) = ukupna greška praćenja kroz vrijeme

$$e_{RMS} = \sqrt{\frac{1}{T} \int_0^T (q(t) - q_d(t))^2 dt}$$

- Mali RMSE = dobro praćenje
- Veliki RMSE = loše praćenje
- Kada koristiti?
 - Primarno kod trajektorija koje se kontinuirano mijenjaju (npr. sinusoidna referenca)
 - Koristan kada želimo procijeniti ukupnu kvalitetu praćenja tijekom cijele simulacije
 - Kod step odziva može se koristiti, ali overshoot, settling time i steady-state error obično daju više informacija

Real2Sim/Sim2Real problem

MY CONTROLLER RUNNING IN MY SIMULATION



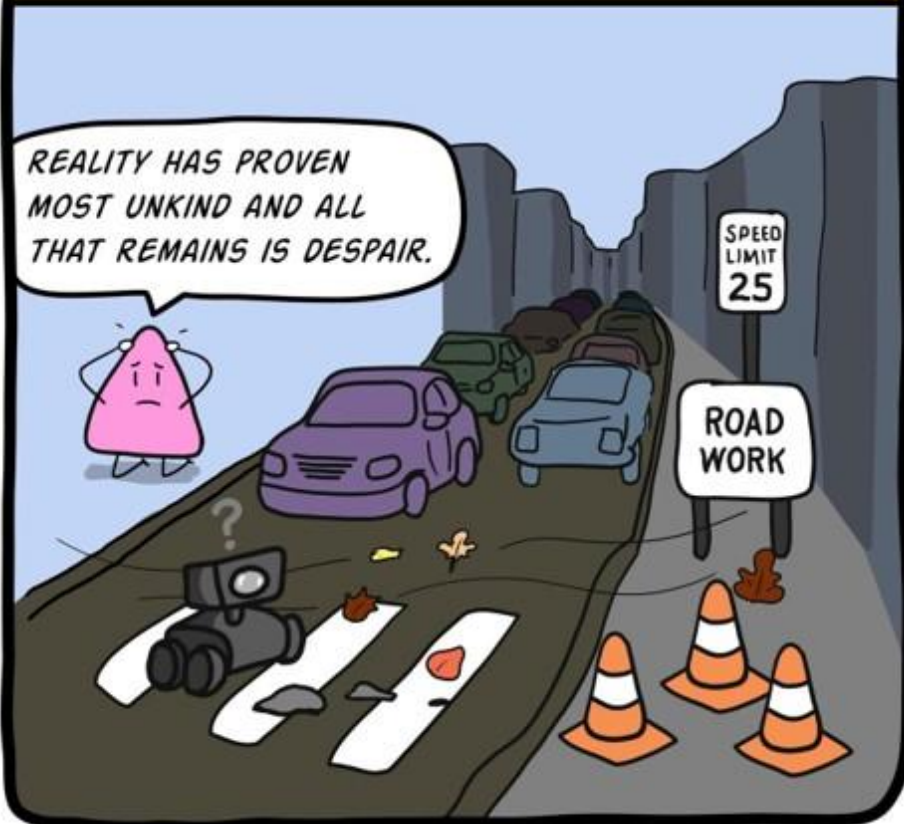
THE EPITOME OF PERFECTION.
NOTHING CAN STOP ME!



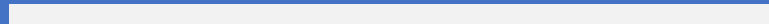
Brian Douglas - EngineeringMedia.com

MY CONTROLLER RUNNING IN THE REAL WORLD

REALITY HAS PROVEN
MOST UNKIND AND ALL
THAT REMAINS IS DESPAIR.



MuJoCo simulator



MuJoCo – simulacijsko okruženje za robotiku

- Fizikalni simulator (physics engine)
- Razvijen za robotiku i biomehaniku
- Simulira:
 - Dinamiku sustava
 - Kontakte (dodiri, sudari)
 - Sile i momenta

The MuJoCo logo is displayed within a dark blue rectangular box. The text 'MuJoCo' is white and uses a bold, sans-serif font. The background of the box has a subtle grid pattern.

MuJoCo

MuJoCo omogućuje realističnu simulaciju gibanja robota prije implementacije na stvarnom sustavu

- Zašto koristimo simulaciju?
 - Testiranje kontrolera (PID tuning) – Real2Sim/Sim2Real problem
 - Sigurnost (nema mogućnosti oštećenja robota)
 - Brže iteracije
 - Jeftina opcija

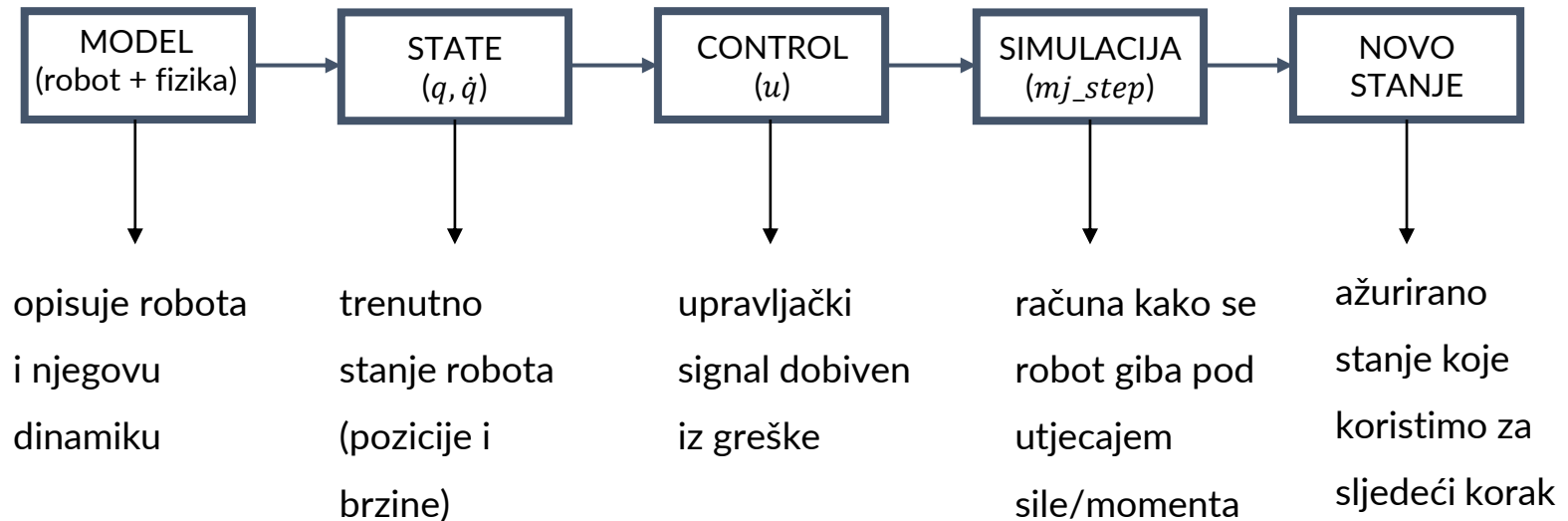
Zašto MuJoCo?

- Stabilna numerika za **nelinearne sustave** (bez divergencije i numeričkih nestabilnosti)
- Podrška za **kompleksne robote** (humanoidi, manipulacija)
- Kontakt → **nelinearnosti** (diskontinuiteti (udarci, trenje), teško modeliranje i upravljanje)
- Reinforcement learning (RL) - treniranje politika bez stvarnog robota (OpenAI Gym/Gymnasium okruženja)
- RL perspektiva: izračun gradijenata, simulacija tisuća epizoda brzo i paralelno

MuJoCo omogućuje učenje i testiranje upravljanja u sustavima gdje su kontakt i nelinearnosti dominantni → HUMANOIDNA ROBOTIKA

MuJoCo overview

- MuJoCo simulacija je petlja: očitamo stanje, izračunamo upravljački signal i sustav evoluira
- Flow:



Ključni objekti u MuJoCo

- **model** - opis sustava (robot, mase, geometrija), definira se kao XML datoteka
- Glavni pojmovi koje koristimo:
 - **joints** (zglobovi):
 - definiraju DOF robota
 - **bodies** (tijela):
 - linkovi robota
 - mase, inercija
 - **geoms** (geometrija):
 - oblik robota (vizualno + kontakt)
 - sudari, dodiri
 - **actuators** (aktuatori):
 - kako šaljemo upravljanje
 - poveznica s data.ctrl
 - **constraints/contacts**:
 - kontakt s okolinom, trenje, sudari

```
1 <mujoco model="3dof_arm">
2
3   <option timestep="0.01" gravity="0 0 -9.81"/>
4
5   <default>
6     <joint damping="1"/>
7     <geom type="capsule" size="0.03 0.3" mass="1"/>
8   </default>
9
10  <worldbody>
11
12    <!-- LINK 1 -->
13    <body name="link1" pos="0 0 0">
14      <joint name="joint1" type="hinge" axis="0 0 1"/>
15      <geom fromto="0 0 0 0 0.6"/>
16
17      <!-- LINK 2 -->
18      <body name="link2" pos="0 0 0.6">
19        <joint name="joint2" type="hinge" axis="0 1 0"/>
20        <geom fromto="0 0 0 0 0.6"/>
21
22        <!-- LINK 3 -->
23        <body name="link3" pos="0 0 0.6">
24          <joint name="joint3" type="hinge" axis="1 0 0"/>
25          <geom fromto="0 0 0 0 0.6"/>
26        </body>
27      </body>
28    </body>
29  </worldbody>
30
31  <actuator>
32    <motor joint="joint1" ctrlrange="-200 200"/>
33    <motor joint="joint2" ctrlrange="-200 200"/>
34    <motor joint="joint3" ctrlrange="-200 200"/>
35  </actuator>
36
37 </mujoco>
```

3DOF primjer s kojim ćemo raditi

Ključni objekti u MuJoCo

```
1 <mujoco model="3dof_arm">
2
3   <option timestep="0.01" gravity="0 0 -9.81"/>
4
5   <default>
6     <joint damping="1"/>
7     <geom type="capsule" size="0.03 0.3" mass="1"/>
8   </default>
9
10  <worldbody>
11
12    <!-- LINK 1 -->
13    <body name="link1" pos="0 0 0">
14      <joint name="joint1" type="hinge" axis="0 0 1"/>
15      <geom fromto="0 0 0 0.6"/>
16
17      <!-- LINK 2 -->
18      <body name="link2" pos="0 0 0.6">
19        <joint name="joint2" type="hinge" axis="0 1 0"/>
20        <geom fromto="0 0 0 0.6"/>
21
22        <!-- LINK 3 -->
23        <body name="link3" pos="0 0 0.6">
24          <joint name="joint3" type="hinge" axis="1 0 0"/>
25          <geom fromto="0 0 0 0.6"/>
26        </body>
27      </body>
28    </body>
29  </worldbody>
30
31  <actuator>
32    <motor joint="joint1" ctrlrange="-200 200"/>
33    <motor joint="joint2" ctrlrange="-200 200"/>
34    <motor joint="joint3" ctrlrange="-200 200"/>
35  </actuator>
36
37 </mujoco>
```

Ime modela

Simulacijski korak 0.01 s,
gravitacija djeluje u negativnom z-smjeru

Svaki zglob ima prigušenje. To smanjuje
oscilacije i "koči" gibanje zglobova.

Svaka geometrija je kapsula, radijusa 0.03
m, pola duljine 0.3 m, mase 1 kg.

'default' –
definiramo
postavke svih
zglobova

Analogno zapisu u ROS-u
Roditeljski odnos zglobova, opis geometrije,
prostornog odnosa linkova

Definicija aktuatora, odnosno načina na koji
šaljemo upravljačke signale robotu.
,ctrlrange' ograničava iznos upravljačkog signala

Čime mi upravljamo?

- Želimo kontrolirati položaj zglobova $q = [q_1, q_2, q_3]$, a željena trajektorija je dana kao $q_d(t)$
- Potrebno je definirati početne uvjete pozicije i brzine zglobova
- Sustavu šaljem upravljački signal u , u MuJoCo sustavu šaljem signal kao: `data.ctrl[:] = u`
- Upravljački signal u računamo pomoću regulatora (P, PD, PI, PID..)
- Kada zadamo upravljački signal za neki tren $u(t)$, MuJoCo prema modelu robota računa dinamiku sustava te daje novo stanje $q, \dot{q}$ preko funkcije `mj_step(model, data)`
- `mj_step` = fizika (uključuje dinamiku sustava, sile, kontakt), funkcija koja poziva direktnu dinamiku robota

Instalacija MuJoCo (Windows)

- Command prompt:
 - `pip install mujoco`
 - `pip install mujoco-python-viewer`
- Ako instalacija ne radi – koristite Python 3.10 i ponovno instalirajte

Zadaci

PRIMJER 1: Implementacija P regulatora

- **Opis zadatka:** Potrebno je implementirati i analizirati djelovanje **proporcionalnog (P) regulatora** za upravljanje 3-DOF robotskim manipulatorom u prostoru zglobova (*joint space*), koristeći zadani simulacijski model u okruženju MuJoCo.

- **Uputa:** Potrebno je simulirati odziv P regulatora za zadano:

1. Početno i željeno stanje sustava

$$q_i = [0, 0, 0], q_f = [1, -0.8, 0.5]$$

2. Interpolacijsku funkciju (5. stupnja polinoma) za generiranje trajektorije

3. Proporcionalni regulator (K_p za svaki zglob iznosi $80 \frac{Nm}{rad}$)

4. Spremanje podataka u `sim_data_primjer1.npz`

5. Vizualni prikaz rezultata

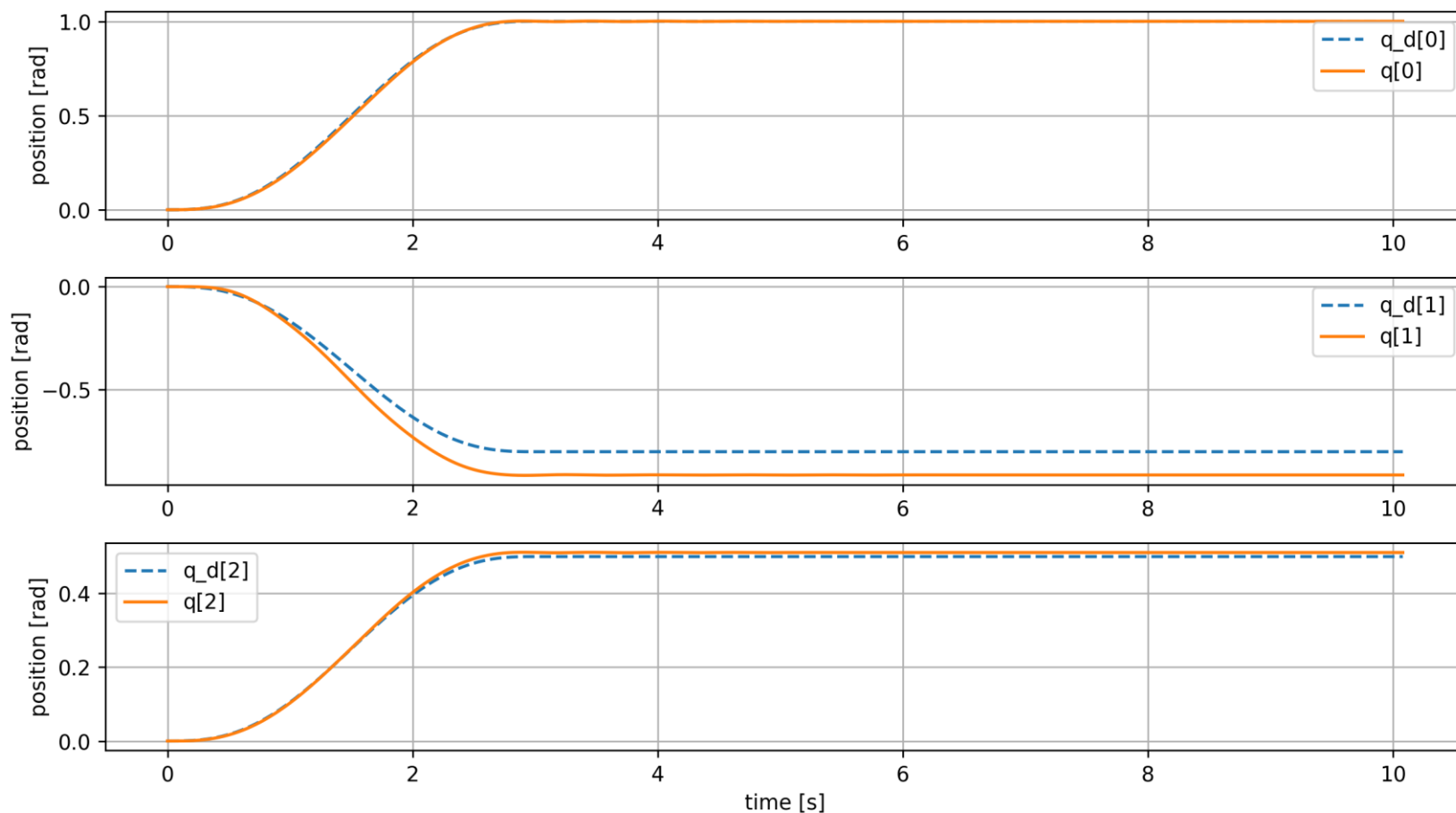
- **Napomena:** na e-učenju nalaze se datoteke koje su rješenja ovog primjera

- `Z_3DOF_P_control.py` – koraci 1.-4.
- `Z_3DOF_plot.py` – korak 5.

PRIMJER 1: Implementacija P regulatora

Rješenja

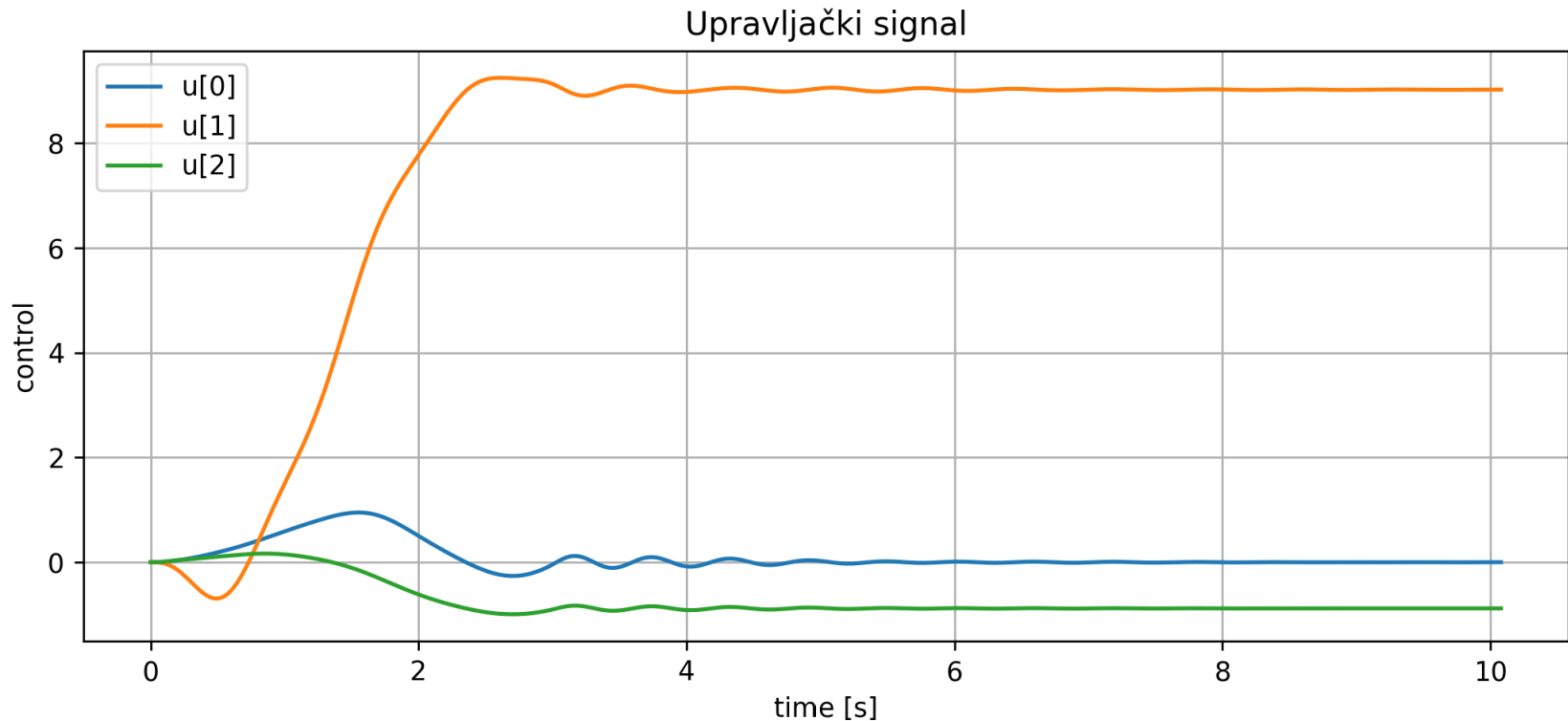
Praćenje trajektorije (P regulator)



- **Napomena:** često je potrebno svakom zglobu namjestiti „njegove” parametre jer se ne ponašaju na isti način

PRIMJER 1: Implementacija P regulatora

Rješenja



- Upravljanje se ne gasi u ustaljenom stanju jer je potrebna kontinuirana kompenzacija (npr. gravitacije)
- Prisutne su male oscilacije u prijelaznom području, što je posljedica dinamike sustava i podešenja regulatora

ZADATAK 1: Implementacija P regulatora

Samostalni zadatak – izvođaču vježbi pokazati rezultate

- **Opis zadatka:** Potrebno je prilagoditi postojeći simulacijski pipeline tako da robot prati **step referencu** u joint space-u.

- **Uputa:** Potrebno je nadopuniti zadani kod implementacijom:

1. Početnog i željenog stanja sustava

$$q_i = [0, 0, 0], q_f = [1, -0.8, 0.5]$$

2. Robot mora pratiti step referencu, vrijeme skoka je 1 sekunda ($t_s = 1.0s$) – hint možete definirati preko jednostavnog 'if' uvjeta

$$q_d(t) = \begin{cases} q_0, & t < t_s \\ q_f, & t \geq t_s \end{cases}$$

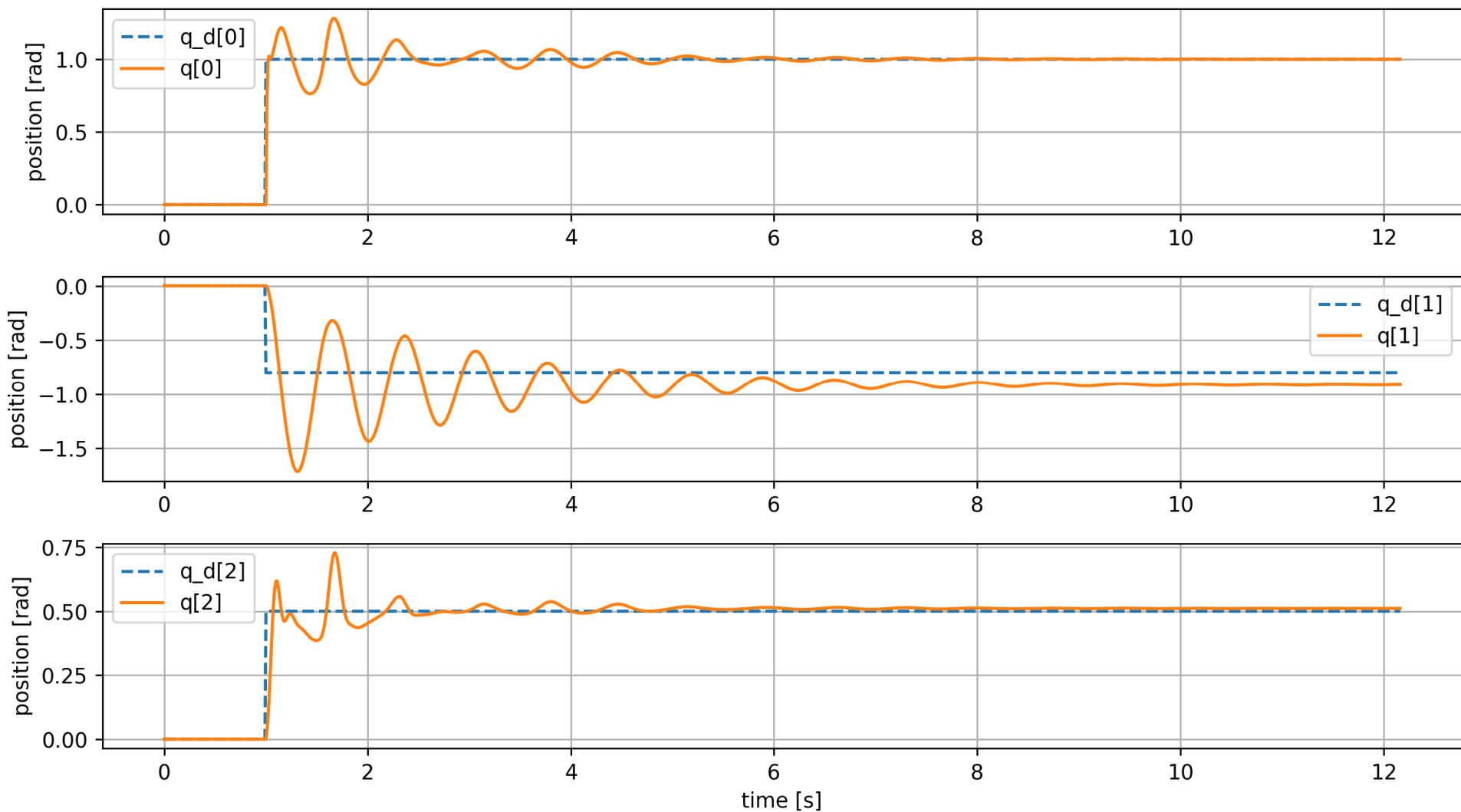
3. Proporcionalnog regulatora (K_p za svaki zglobov iznosi $80 \frac{Nm}{rad}$)
4. Spremanje podataka u `sim_data_zadatak1.npz`
5. Vizualni prikaz rezultata
6. Izračunati relevantne metrike odziva na referencu (overshoot, settling time, steady-state error)

- **UPUTA:** Simulaciju zaustavite kada robot prestane oscilirati (2-3 sekunde nakon).

ZADATAK 1: Implementacija P regulatora, step

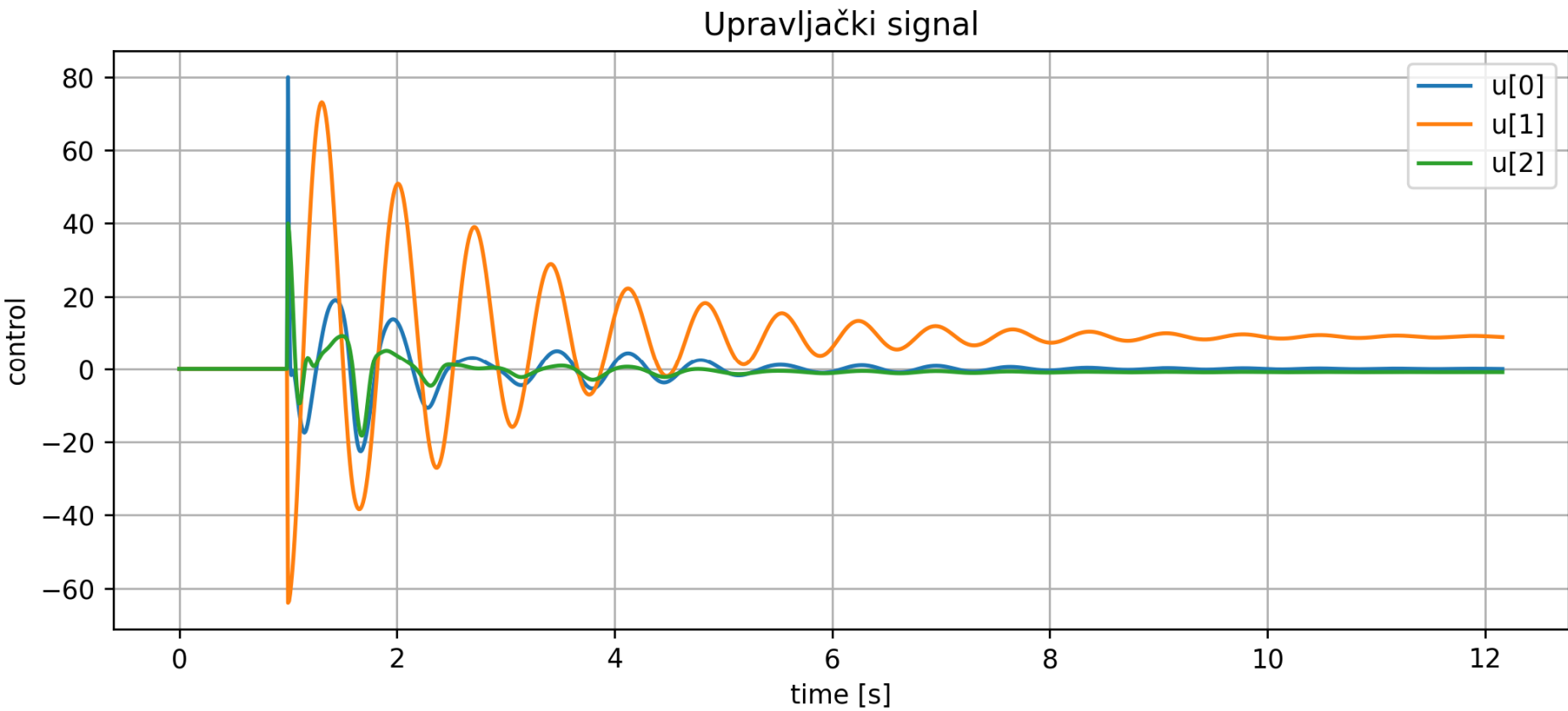
Rješenja

Praćenje trajektorije (P regulator)



ZADATAK 1: Implementacija P regulatora, step

Rješenja



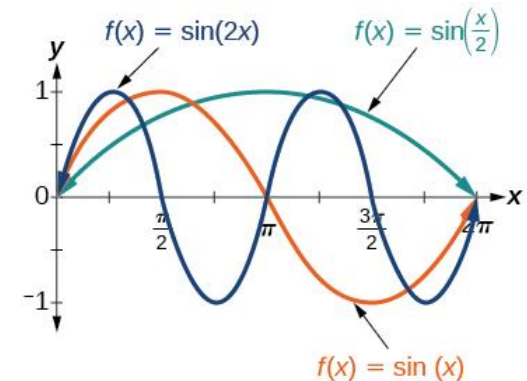
ZADATAK 2: Implementacija P regulatora

Samostalni zadatak – izvođaču vježbi pokazati rezultate

- **Opis zadatka:** Potrebno je prilagoditi postojeći simulacijski pipeline tako da robot prati sinusoidnu referencu u joint space-u.
- **Uputa:** Potrebno je nadopuniti zadani kod implementacijom:
 1. Robot mora pratiti sinusoidnu referencu, sa sljedećim parametrima: $A = [0.5, 0.4, 0.3]$, $f = 0.5\text{Hz}$, srednja konfiguracija (oko koje zakon oscilira) - $q_c = [0, -0.3, 0.2]$

$$q_d(t) = q_c + A \sin(2\pi f t)$$

2. Proporcionalnog regulatora (K_p za svaki zglob iznosi $80 \frac{\text{Nm}}{\text{rad}}$)
3. Spremanje podataka u `sim_data_zadatak2.npz`
4. Vizualni prikaz rezultata
5. Izračunati relevantnu metriku odziva na referencu (RMSE)

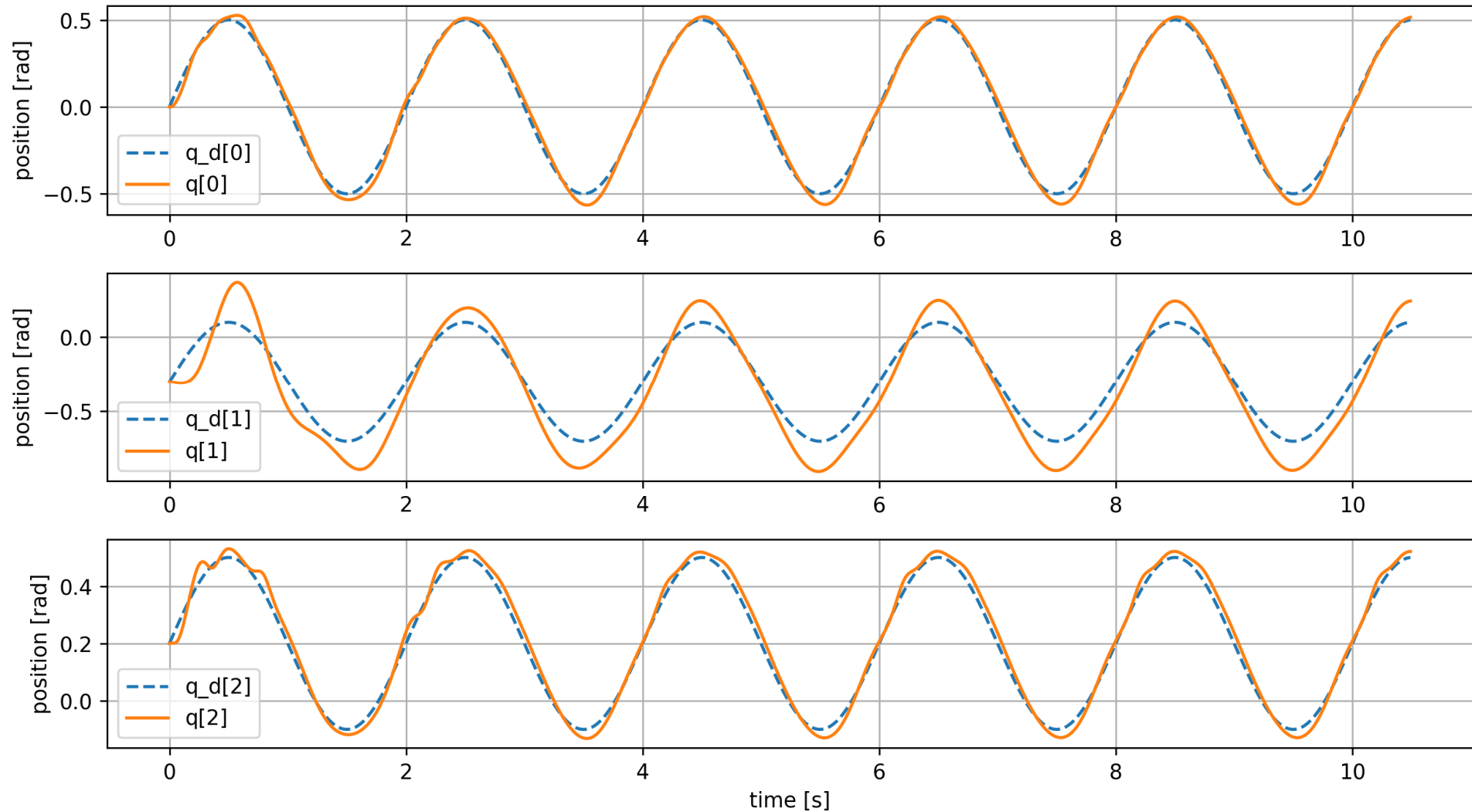


- **UPUTA:** Simulaciju zaustavite nakon cca. 10 sekundi.

ZADATAK 2: Implementacija P regulatora

Rješenja

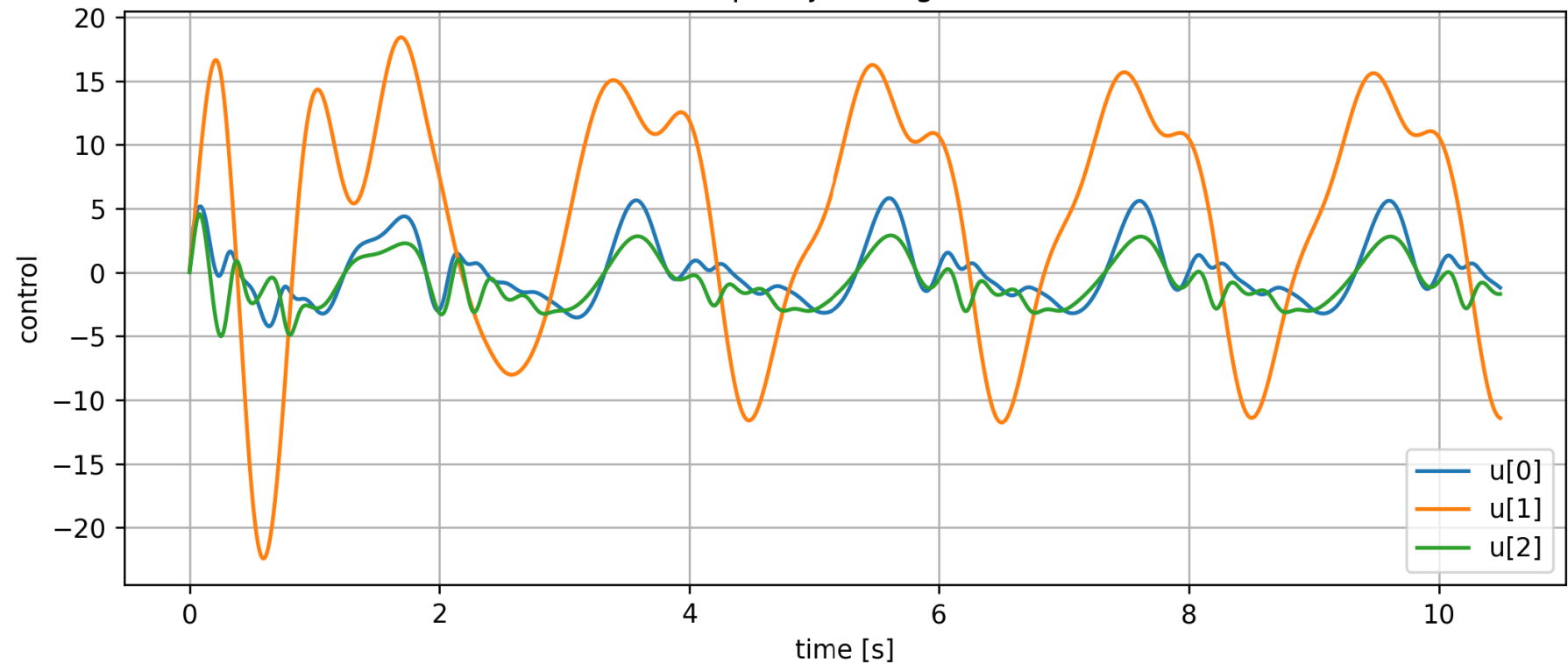
Praćenje trajektorije (P regulator)



ZADATAK 2: Implementacija P regulatora

Rješenja

Upravljački signal



ZADATAK 3: Implementacija PD regulatora

Samostalni zadatak – izvođaču vježbi pokazati rezultate

- **Opis zadatka:** Potrebno je proširiti postojeći simulacijski pipeline tako da robot prati odabranu referencu u joint space-u pomoću **PD regulatora**.
- **Uputa:** Potrebno je nadopuniti zadani kod implementacijom:
 1. Odabir referentne trajektorije - potrebno je odabrati jednu od prethodno korištenih referenci (step ili sinusoidna referenca)
 2. Dodavanje derivacijskog člana
 3. Proporcionalno pojačanje K_p može ostati jednako kao u prethodnim zadacima, Vrijednost derivacijskog pojačanja K_d potrebno je odabrati eksperimentalno tako da se postigne manje oscilacija, manji overshoot te brže smirivanje sustava
 4. Spremanje podataka u `sim_data_zadatak3_$vrsta_reference$.npz`
 5. Vizualni prikaz rezultata
 6. Izračunati relevantnu metriku odziva na referencu
- **HINT:** POSTAVITI RAZLIČITE VRIJEDNOSTI POJAČANJA K_d ZA POJEDINI ZGLOB.
- **HINT:** $\dot{q}(t = 0) = 0$, a trenutno stanje brzine čitamo analogno poziciji (`data.qvel.copy()`)
- **HINT:** potrebno je definirati zasebno grešku te brzinu greške

ZADATAK 4: Implementacija PD regulatora uz disturbance

Samostalni zadatak – izvođaču vježbi pokazati rezultate

- **Opis zadatka:** Potrebno je nadograditi postojeći PD regulator uvođenjem **vanjske smetnje (disturbance)** te analizirati njen utjecaj na ponašanje sustava.
- **Uputa:** Potrebno je nadopuniti zadani kod implementacijom:
 1. Odabir referentne trajektorije - potrebno je odabrati jednu od prethodno korištenih referenci (step ili sinusoidna referenca) – istu kao u prošlom zadatku
 2. Proporcionalno pojačanje K_p i derivativno pojačanje K_d moraju ostati jednaki kao u prošlom zadatku.
 3. Uvođenje modela smetnje za sljedeće iznose: $d = [5.0, 5.0, 5.0]Nm$
 4. Spremanje podataka u `sim_data_zadatak4_$vrsta_reference$.npz`
 5. Vizualni prikaz rezultata
 6. Izračunati relevantnu metriku odziva na referencu
- **HINT:** dodavanje smetnje uz upravljački signal (mi SIMULIRAMO smetnju kao dio upravljačkog signala)

$$u_{total} = u(t) + d$$

ZADATAK 5: Implementacija PID regulatora

Samostalni zadatak – izvođaču vježbi pokazati rezultate

- **Opis zadatka:** Potrebno je proširiti postojeći PD regulator tako da se smanji ili ukloni **steady-state error** pri praćenju odabrane reference u joint space-u pomoću **PID regulatora**.
- **Uputa:** Potrebno je nadopuniti zadani kod implementacijom:
 1. Odabir referentne trajektorije - potrebno je odabrati jednu od prethodno korištenih referenci (step ili sinusoidna referenca) – istu kao u prošlom zadatku
 2. Dodavanje integralnog člana
 3. Proporcionalno pojačanje K_p i derivativno pojačanje K_d mogu ostati jednaki kao u prethodnom zadatku. Vrijednost integralnog pojačanja K_i potrebno je odabrati eksperimentalno tako da se ukloni steady-state error izazvan simuliranom smetnjom, uz zadržavanje stabilnosti sustava. Ovime simuliramo nesavršenosti i nelinearnosti sustava.
 4. Spremanje podataka u `sim_data_zadatak5_${vrsta_reference}.npz`
 5. Vizualni prikaz rezultata
 6. Izračunati relevantnu metriku odziva na referencu
- **HINT:** potrebno je definirati zasebno grešku, derivaciju (brzinu) greške te integral greške

ZADATAK 6: PID u prisutnosti white noise-a

Samostalni zadatak – izvođaču vježbi pokazati rezultate

- **Opis zadatka:** Potrebno je nadograditi postojeći PID regulator dodavanjem white noise komponente u mjerenje položaja zglobova te analizirati utjecaj šuma na ponašanje sustava.
- **Uputa:** Potrebno je nadopuniti zadani kod implementacijom:
 1. Odabir referentne trajektorije - potrebno je odabrati jednu od prethodno korištenih referenci (step ili sinusoidna referenca) – istu kao u prošlom zadatku
 2. Pojačanja K_p , K_d i K_i mogu ostati jednaki kao u prethodnom zadatku. Prema potrebi, u cilju dobivanja boljeg odziva, možete promijeniti parametre.
 2. Potrebno je implementirati white noise koji simulira šum u mjerenjima položaja zglobova, nesavršenosti senzora, tj. realne uvjete rada robotskog sustava. Standardna devijacija šuma neka bude 0.02
 3. Spremanje podataka u `sim_data_zadatak6_${vrsta_reference}.npz`
 4. Vizualni prikaz rezultata
 5. Izračunati relevantnu metriku odziva na referencu
- **HINT:** šum se dodaje na izmjerenu vrijednost položaja: $q_{meas} = q + n$ (q_{meas} = konačni izmjereni položaj, q je stvarno izmjeren podatak iz MuJoCo, n = šum)
- **HINT:** White noise generira se iz normalne distribucije
- **HINT:** Regulator koristi šumovito mjerenje za estimaciju greške

Upute za predaju izvješća s vježbe

- Izvješće s vježbe treba sadržavati oba plota (praćenje reference te upravljački zakon) za sva tri zgloba za sve zadatke
- Prema tome što ste odabrali kao referencu (step ili sinus), potrebno je napraviti tabličnu usporedbu **svih zadataka**
- Broj metrika ovisi o odabranoj referenci
- Rok predaje: 8.5., 16:00. Predaja izvješća preko e-učenja.

JOINT	ZADATAK	REFERENCA	POJAČANJA	METRIKA 1	METRIKA 2	METRIKA 3
1	1					
2	1					
3	1					
1	2					
2	2					
3	2					
⋮	⋮	⋮	⋮	⋮	⋮	⋮

- **NAPOMENA:** Pazite da su vremena simulacije približno ista.